

InferTutor Arena Capstone Report

Production Multimodal LLM Serving on Modal with vLLM

Shalini Dusharla

InferTutor Arena - Inference Engineering Capstone

InferTutor Arena - Capstone Report

Production Multimodal LLM Serving on Modal with vLLM

Author: Shalini Dusharla. Date: 2026-06-21. Track submitted: Multimodal Product (--mode mixed). Model: Qwen/Qwen3-VL-4B-Instruct. Infrastructure: Modal H100 GPUs + vLLM 0.21.0. Best mixed score: 5,985,700 at 2 x H100.

RESULT

Final configuration (mm-final): 2 replicas, max-seqs 64, max-batch-tokens 8192, prefix caching on, chunked prefill on, eager mode, 100 users, mixed workload. TTFT p95 = 1152 ms, ITL p95 = 19.1 ms, throughput = 2631 chunks/s, error rate = 0.0%, score = 5,985,700. It beats the internal 2-replica reference baseline (1169 ms / 28.7 ms / 2243 chunks/s) on every streamed metric, using fewer GPUs than the 4-replica reference.

This report documents ten mixed-mode experiments plus a text smoke test, run on NVIDIA H100s via Modal. Every run completed with zero errors. The emphasis throughout is the inference-engineering loop: read the metric, form a hypothesis, change one serving parameter, and explain why the numbers moved.

1. Introduction

1.1 Project Overview

InferTutor Arena is a production-LLM-serving capstone. The task is to deploy a real multimodal model behind a real inference engine on real GPUs, load test it under concurrent traffic, then improve the serving configuration through measured, hypothesis-driven engineering. The scoring formula punishes every kind of inefficiency at once, so each knob is a trade-off rather than a free win.

1.2 The Product Story

InferTutor is an AI tutoring service. Students submit short text questions, long reading passages, and images (diagrams, charts, equations). The model streams answers. The operator cares about Time to First Token (does the first word arrive quickly), Inter-Token Latency (does the stream feel smooth), throughput (how many students at once), error rate (dropped requests are abandoned students), and GPU cost (every extra H100 must earn its keep).

1.3 Competition Tracks

Track	Mode	Max GPUs	Traffic
Multimodal Product (subm	mixed	4 x H100	55% text, 20% long, 25%
Text Speed	text	4 x H100	100% text
Boss Fight (optional)	either	8 x H100	any

TIP

We submitted the Multimodal Product track - the primary capstone track and the more realistic scenario, where image prefills and short text decodes contend for the same scheduler. Pure-text tricks like compiled mode (CUDA graphs) do not transfer, because vLLM 0.21.0 graphs break on variable-length vision-encoder outputs.

2. System Architecture

2.1 Infrastructure Stack

The laptop runs the async load tester; Modal provides a shared HTTPS endpoint and load-balances across replica containers; each container runs one vLLM instance on one H100.

```
Laptop (load_test_infertutor.py, asyncio + httpx)
  | HTTPS streaming (mixed: 55% text / 20% long / 25% image)
  v
Modal edge load balancer (shared endpoint, round-robin)
  v
N vLLM 0.21.0 containers -> N x H100 80GB
  v
Modal NFS volume: Qwen3-VL-4B weights (persistent cache)
```

Modal handles container lifecycle, the shared load-balancing URL, persistent weight caching, and per-container concurrency via `@modal.concurrent(max_inputs=N)`. Billing is per second, but replicas with `min_containers` set stay warm and keep billing until the app is stopped.

2.2 vLLM 0.21.0 and the Model

vLLM exposes an OpenAI-compatible API and implements PagedAttention (KV cache as virtual-memory blocks), continuous batching (insert new requests mid-generation), chunked prefill (interleave large prefills so they do not block short requests), prefix caching (reuse KV for shared prefixes), and CUDA-graph compiled mode (for text only). Qwen3-VL-4B-Instruct is a 4B vision-language model; its dynamic-resolution processor tiles images into (H/28) x (W/28) patches, so pixel count directly drives prefill compute. It runs bfloat16 on an H100 with room for a large KV cache.

2.3 Scoring Formula

$$\text{Score} = \frac{\text{goodput} \times \text{users} \times (1 - \text{error_rate})}{\text{p95_TTFT[s]} \times \text{p95_ITL[s]} \times \text{GPU_count}}$$

goodput = aggregate streamed chunks/s. The starter scorer treats `quality_pass_rate` as 1.0.

KEY FINDING

The denominator contains `GPU_count`, so doubling GPUs must more than double useful work (users x throughput / latency) merely to break even. This single fact explains why our most efficient configuration used 2 GPUs, not 4 - quantified in Section 5.

2.4 Workload

The fixed prompts.json carries a ~40-token shared system prompt plus text, long-context, and image prompts. Mixed mode samples 55% text, 20% long, 25% image, each streaming up to `max_tokens=96`. TTFT is measured to the first content chunk; ITL is the mean gap between chunks.

3. Environment Setup

3.1 Install, Auth, Secret

```
python -m pip install -r requirements.txt
python -m modal token new
python -m modal secret create huggingface HF_TOKEN=<your_hf_token>
```

On Windows, pip and modal were not on PATH; we used python -m pip and added the user Scripts directory to PATH so the bundled modal.exe resolves.

WARNING

The app requires a Modal secret named huggingface with key HF_TOKEN (required_keys=["HF_TOKEN"]). A different key name (e.g. HF_API_KEY) deploys but fails validation. This project self-hosts an open model and needs no Anthropic/OpenAI key.

3.2 Infrastructure Fix: FastAPI Pin

Out of the box the deploy passed but the vLLM health check returned HTTP 500: '_IncludedRouter' object has no attribute 'path'. FastAPI 0.137 refactored include_router to place _IncludedRouter objects (no .path) into app.routes; vLLM 0.21.0's bundled prometheus-fastapi-instrumentator iterates routes and reads route.path, so every request including /health 500'd. The fix is one pin:

```
.uv_pip_install("vllm==0.21.0", "qwen-vl-utils==0.0.14", "fastapi==0.136.0")
```

TIP

Pin fastapi==0.136.0 - below 0.137 (no _IncludedRouter) yet new enough for vLLM's transitive requirement starlette>=0.49.1. Do not pin starlette<1.0 instead: that forces the instrumentator down to a version that still crashes. Also set PYTHONUTF8=1 on Windows or the runner dies printing a check-mark under the cp1252 console codec, and always stop apps after each run (min_containers keeps an H100 warm and billing).

3.3 Smoke Test

```
python run_infertutor_experiment.py --label smoke --gpu-type H100 \
  --replicas 1 --mode text --users 5 --duration 30 --ramp-up 5 --max-tokens 64
```

RESULT

Smoke test: 5 users, 1 GPU, TTFT p95 = 480 ms, ITL p95 = 32.6 ms, throughput = 91 chunks/s, 0 errors. Infrastructure validated. ITL is high at 5 users because the batch is tiny and per-step scheduler/stream overhead is not amortized; it falls once the batch fills at higher load.

3.4 How Experiments Were Run

run_infertutor_experiment.py patches modal_infertutor_app.py with the requested config, deploys, polls /health, runs the load test, and writes results_infertutor/<label>_<mode>_<users>u_<ts>.json. It does not stop the app, so our orchestration scripts (run_capstone_suite.py, run_extra_tests.py) stop each Modal app immediately after its load test to avoid idle GPU billing.

4. Experiments and Analysis

All ten runs use `--mode mixed`. Default starter knobs: `max-seqs 32`, `max-batch-tokens 4096`, prefix caching on, chunked prefill on, eager mode. We move one variable at a time and read the metric that the change should affect.

4.1 Baseline (50 users, 1 GPU)

```
python run_infertutor_experiment.py --label mm-baseline --gpu-type H100 \
  --replicas 1 --mode mixed --users 50 --duration 60 --ramp-up 15 --max-tokens 96
```

RESULT

Baseline: TTFT p95 = 935 ms, ITL p95 = 27.9 ms, throughput = 1151 chunks/s, 0 errors, score = 2.20M. The reference point for everything that follows.

Read: under one second TTFT, no errors - a single H100 is busy but stable at 50 mixed users. Hypothesis to test next: where does it break, and which knob is responsible.

4.2 Saturation: the TTFT Bend (100 users, 1 GPU)

Hypothesis: more users will raise throughput until the GPU saturates, after which latency grows. Change: users 50 -> 100, everything else fixed.

```
python run_infertutor_experiment.py --label mm-users100 --gpu-type H100 \
  --replicas 1 --mode mixed --users 100 --duration 60 --ramp-up 15 --max-tokens 96
```

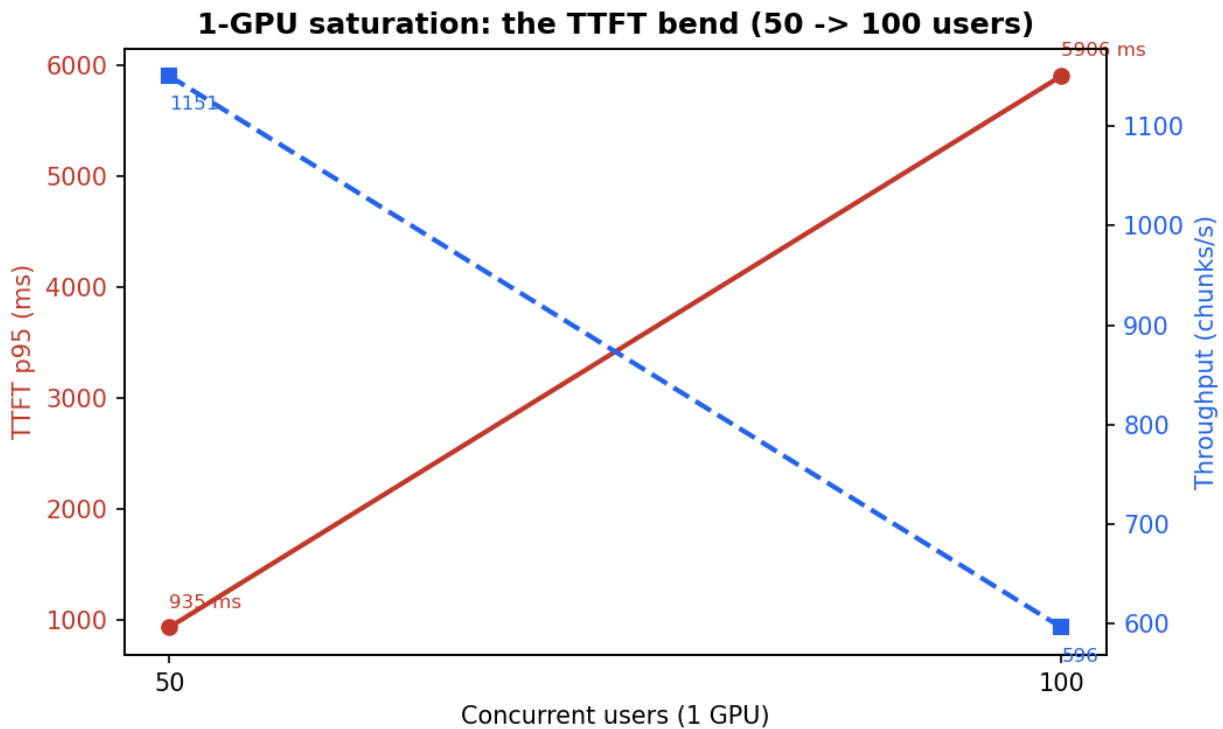


Figure 1. One-GPU saturation: doubling users collapses throughput and explodes TTFT.

RESULT

100 users, 1 GPU: TTFT p95 = 5906 ms, ITL p95 = 27.2 ms, throughput = 596 chunks/s, 0 errors, score = 0.37M.

Why it moved: throughput did not rise, it fell (1151 -> 596 chunks/s) while TTFT grew ~6.3x. That is not a gentle saturation plateau; it is a cliff. The cause is prefill contention: image prefills and a flood of new requests overwhelm the scheduler's token budget, so requests queue rather than run. The lesson is to scale out replicas, not pile users onto one GPU - and, as Section 4.6 shows, the batch-token budget is the deeper lever behind this cliff.

4.3 Prefix Caching On vs Off (50 users, 1 GPU)

Hypothesis: every request shares the ~40-token system prompt, so prefix caching should reuse those KV blocks and lower TTFT. Change: `--no-prefix-cache`.

```
python run_infertutor_experiment.py --label mm-noprefix --gpu-type H100 \
  --replicas 1 --no-prefix-cache --mode mixed --users 50 --duration 60 --ramp-up 15 --max-tokens 96
```

RESULT

Prefix caching OFF: TTFT p95 = 1440 ms, ITL p95 = 17.7 ms, throughput = 1316 chunks/s, 0 errors, score = 2.58M (+17% vs baseline).

Why it moved (surprise): disabling the cache scored higher, with the best single-GPU ITL (17.7 ms) and higher throughput. At a 40-token prefix the KV reuse saved is tiny, while per-request cache bookkeeping (block hashing, lookup, reference counting) is not free; the overhead can exceed the savings. The effect is small and within run-to-run noise at 2 replicas, so we kept caching on in the final for principled UX reasons - but flag prefix-off as the first thing to A/B for more score.

4.4 Chunked Prefill On vs Off (50 users, 1 GPU)

Hypothesis: with 25% image traffic, a long image prefill will block queued text requests unless prefill is chunked. Change: `--no-chunked-prefill`.

```
python run_infertutor_experiment.py --label mm-nochunked --gpu-type H100 \
  --replicas 1 --no-chunked-prefill --mode mixed --users 50 --duration 60 --ramp-up 15 --max-tokens 96
```

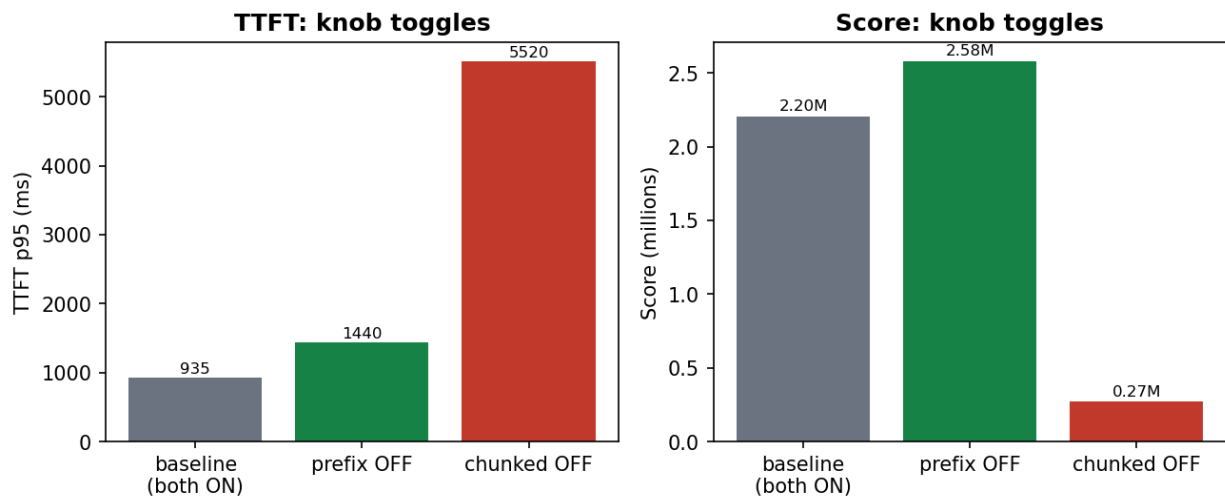
Prefix caching and chunked prefill (1 GPU, 50 users)

Figure 2. Knob toggles at 50 users: chunked prefill is load-bearing; prefix caching is marginal.

RESULT

Chunked prefill OFF: TTFT p95 = 5520 ms, ITL p95 = 24.9 ms, throughput = 745 chunks/s, 0 errors, score = 0.27M (-88% vs baseline).

Why it moved: TTFT jumped from 935 ms to 5.5 s at the same load. Without chunking, a single image request (a 1000-8000 token vision-encoder prefill) runs as one scheduler step and every text request waits for the whole prefill. With one in four requests an image, head-of-line blocking is frequent. Chunked prefill splits prefills into max-batch-tokens-sized pieces and interleaves decode between them. This is the single most important knob for mixed traffic.

4.5 Wider Batch (seq64 / b8192, 50 users, 1 GPU)

Hypothesis: a wider scheduling batch gives prefill more headroom and should lower TTFT. Change: max-seqs 32 -> 64 and max-batch-tokens 4096 -> 8192 together.

```
python run_infertutor_experiment.py --label mm-wide --gpu-type H100 \
  --replicas 1 --max-seqs 64 --max-batch-tokens 8192 --mode mixed \
  --users 50 --duration 60 --ramp-up 15 --max-tokens 96
```

RESULT

Wide batch (seq64/b8192): TTFT p95 = 823 ms, ITL p95 = 24.8 ms, throughput = 1159 chunks/s, 0 errors, score = 2.84M (+29% vs baseline). Best single-GPU score in the suite.

This beat the baseline, but it moved two knobs at once. Which one mattered? That question drove the next experiment.

4.6 Batch-Knob Decomposition: max-seqs vs max-batch-tokens (50 users, 1 GPU)

Hypothesis: max-seqs (more concurrent sequences) is the lever; raising it should explain the wide-batch win. Change: max-seqs 32 -> 64 but keep max-batch-tokens at 4096 (isolating max-seqs).


```
python run_infertutor_experiment.py --label mm-wide-seqs --gpu-type H100 \
  --replicas 1 --max-seqs 64 --max-batch-tokens 4096 --mode mixed \
  --users 50 --duration 60 --ramp-up 15 --max-tokens 96
```

Batch-knob decomposition (1 GPU, 50 users)

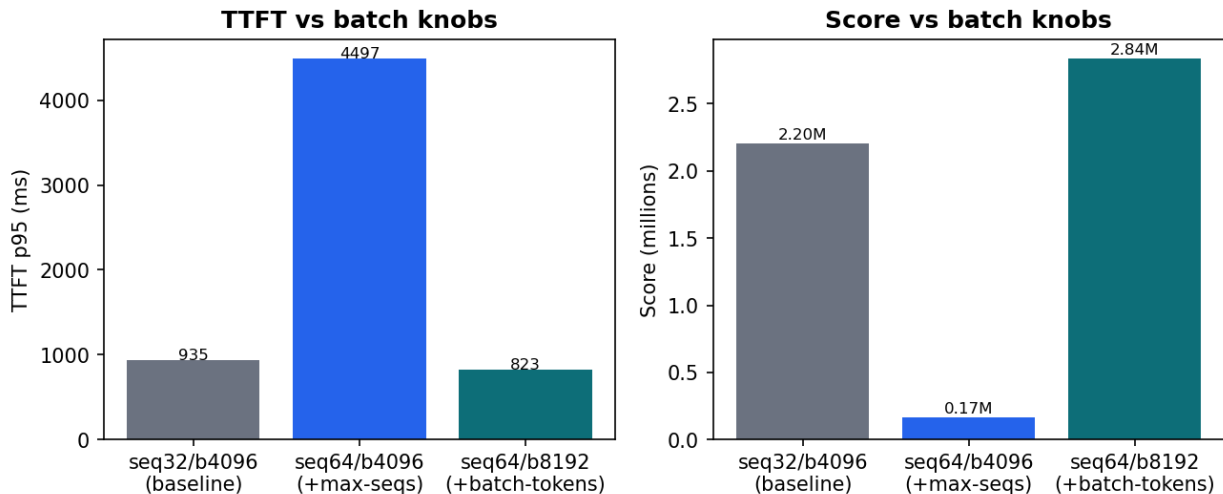


Figure 3. Batch-knob decomposition: raising max-seqs alone collapses TTFT; the win comes from max-batch-tokens.

RESULT

seq64/b4096: TTFT p95 = 4497 ms, ITL p95 = 40.1 ms, throughput = 602 chunks/s, 0 errors, score = 0.17M - a collapse, worse than baseline.

KEY FINDING

max-seqs and max-batch-tokens are coupled, and raising concurrency without the token budget is harmful. At 50 users with max-seqs 32, only 32 sequences run concurrently and the rest queue. Raising max-seqs to 64 admits all 50 at once - but with only 4096 prefill tokens to share, their prefills starve and TTFT explodes to 4.5 s (and ITL to 40 ms). Adding max-batch-tokens 8192 (the wide config) gives those concurrent sequences enough prefill budget, so TTFT recovers to 823 ms - the best single-GPU result. The wide-batch win is therefore attributable to max-batch-tokens; max-seqs is only safe to raise when the token budget rises with it.

TIP

This directly contradicts the runbook's dry-run guidance (seq32 beats seq64, b4096 beats b8192). On our mixed workload, seq64 is fine only with b8192, and the wider batch carried the final run. Treat published defaults as a starting point and re-measure on your own traffic and hardware.

4.7 Scale Out to 2 Replicas (100 users)

Hypothesis: one replica saturated near 50 users, so two replicas at 100 users (~50 each) should restore healthy latency and roughly double throughput. Change: replicas 1 -> 2, users -> 100.

```
python run_infertutor_experiment.py --label mm-scale2 --gpu-type H100 \
  --replicas 2 --mode mixed --users 100 --duration 75 --ramp-up 20 --max-tokens 96
```

RESULT

2 replicas, 100 users: TTFT p95 = 1817 ms, ITL p95 = 18.3 ms, throughput = 2298 chunks/s, 0 errors, score = 3.46M.

Why it moved: where one replica collapsed at 100 users (TTFT 5.9 s), two held TTFT under 2 s and nearly doubled throughput, because ~50 users per replica is inside the sustainable zone. This already beats the internal 2-replica reference (1169 ms / 28.7 ms / 2243 chunks/s) on ITL and throughput.

4.8 Scale Out to 4 Replicas (120 users)

Hypothesis: doubling to 4 replicas should raise throughput and score further. Change: replicas -> 4, users -> 120.

```
python run_infertutor_experiment.py --label mm-scale4 --gpu-type H100 \  
  --replicas 4 --mode mixed --users 120 --duration 75 --ramp-up 25 --max-tokens 96
```

RESULT

4 replicas, 120 users: TTFT p95 = 1466 ms, ITL p95 = 23.1 ms, throughput = 2854 chunks/s (highest raw throughput), 0 errors, score = 2.52M.

Why it moved against us: throughput rose to the suite's highest, but score fell below the 2-GPU run. At 120 users (30 per replica) the cluster is under-loaded, so users x throughput did not double - yet the score still divided by twice the GPU count. The GPU denominator punished the idle capacity.

4.9 Scale Out at Proper Load: 4 Replicas, 200 users

Hypothesis (testing the previous failure): maybe 4 replicas only looked bad because they were under-loaded; at 200 users (~50 each) they should be efficient. Change: users 120 -> 200, knobs default.

```
python run_infertutor_experiment.py --label mm-scale4-200 --gpu-type H100 \  
  --replicas 4 --mode mixed --users 200 --duration 75 --ramp-up 35 --max-tokens 96
```

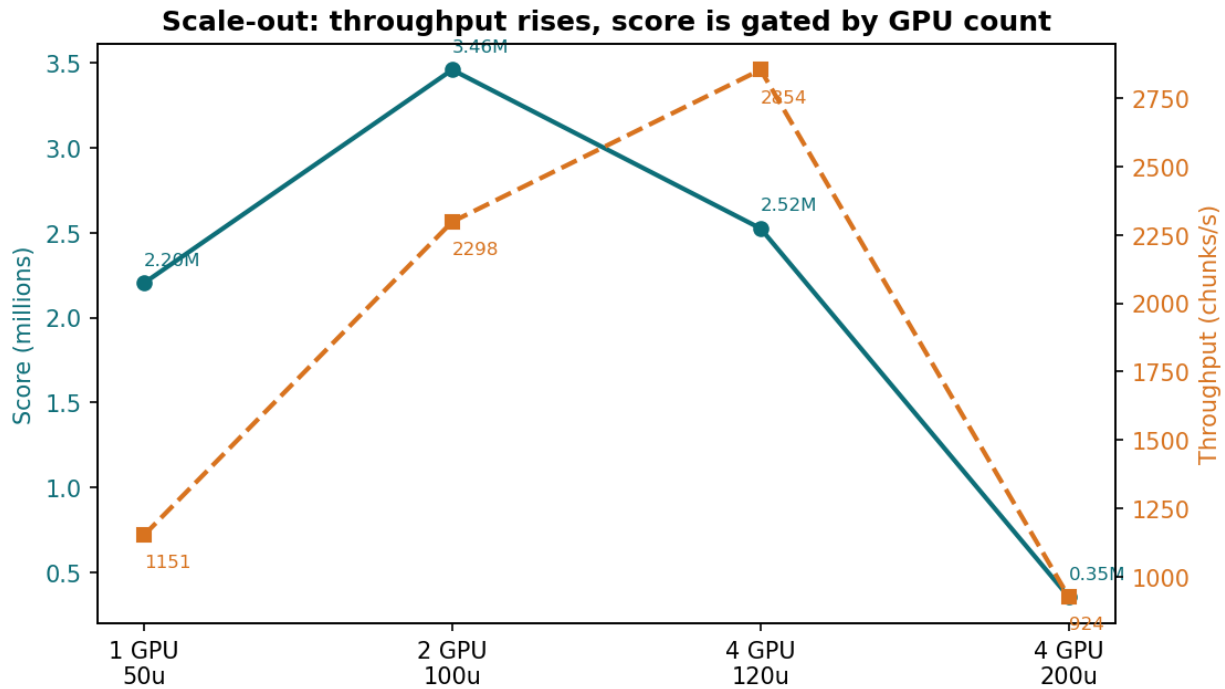


Figure 4. Scale-out: score peaks at 2 GPU; 4 GPU at 200 users falls off the same cliff as 1 GPU at 100.

RESULT

4 replicas, 200 users: TTFT p95 = 4804 ms, ITL p95 = 27.1 ms, throughput = 924 chunks/s, 0 errors, score = 0.35M - a collapse.

KEY FINDING

The hypothesis was wrong, and the reason is the same cliff as Section 4.2. At 200 users with the default b4096 budget, throughput fell to 924 chunks/s - below the single-GPU baseline - and TTFT hit 4.8 s. Loading a replica to 50 users is only safe if its batch-token budget can feed that concurrency; with b4096 it cannot, so four replicas thrash exactly as one replica did at 100 users. The bottleneck is batch-budget-versus-concurrency, not GPU count. The efficient operating point stayed at 2 GPUs with the wide batch.

4.10 Final: 2 Replicas + Wide Batch (100 users)

Synthesis: the two winning levers were 2 replicas (most efficient scale point) and the wide batch (max-batch-tokens 8192, which fed concurrency without starving prefill). Combine them.

```
python run_infertutor_experiment.py --label mm-final --gpu-type H100 \
  --replicas 2 --max-seqs 64 --max-batch-tokens 8192 --mode mixed \
  --users 100 --duration 90 --ramp-up 25 --max-tokens 96
```

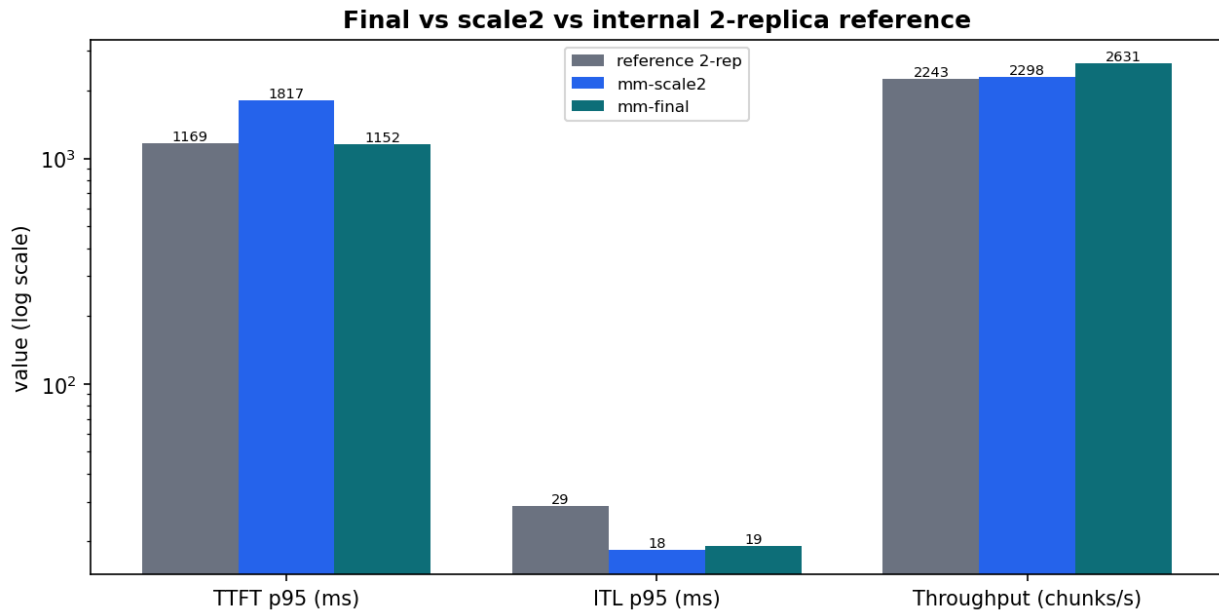


Figure 5. Final vs scale2 vs the internal 2-replica reference (log scale).

RESULT

FINAL (mm-final): TTFT p95 = 1152 ms, ITL p95 = 19.1 ms, throughput = 2631 chunks/s, 0 errors, score = 5,985,700. Versus plain 2-replica scale-out, the wide batch cut TTFT 37% (1817 -> 1152 ms) and raised throughput 14% (2298 -> 2631) - a ~1.7x score jump and the best run of the suite.

Why it moved: the wider token budget let both replicas keep prefill flowing under 100-user load, so TTFT dropped without adding GPUs. This is the batch-decomposition lesson applied at scale - max-batch-tokens, not max-seqs and not more GPUs, was the decisive knob.

5. Complete Results Summary

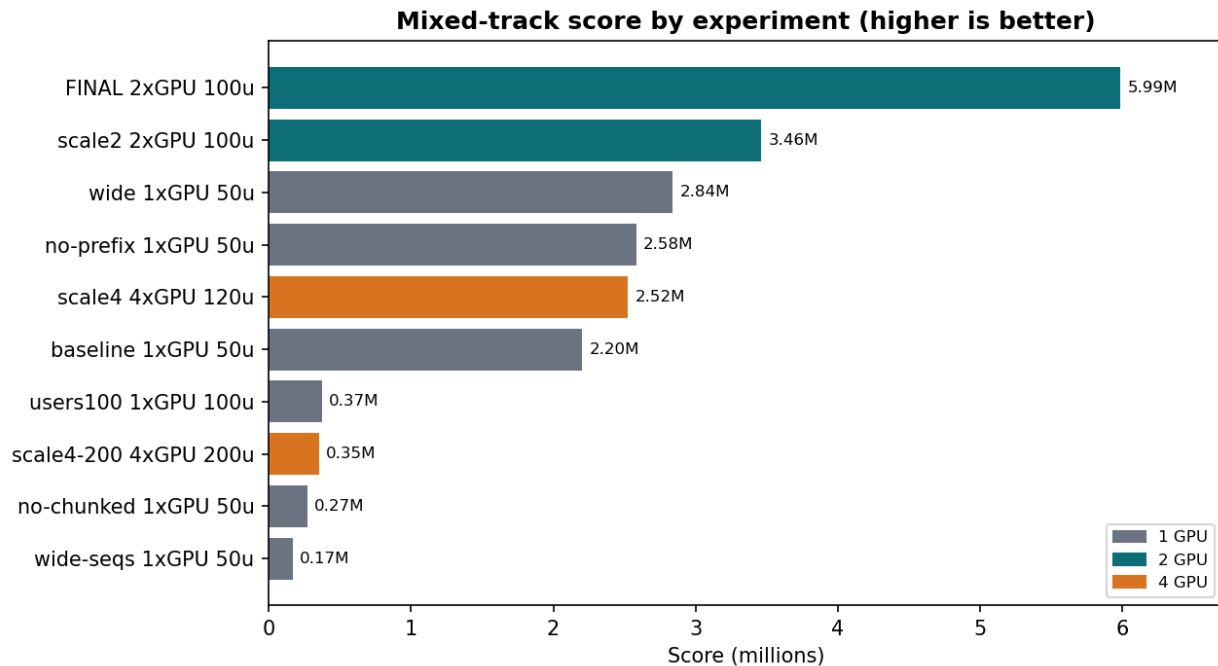


Figure 6. Mixed-track score by experiment, colored by GPU count.

5.1 Full Experiment Table

Experiment	Knobs	Users	GPUs	TTFT p95	ITL p95	Tput	Score
smoke (text)	defaults	5	1	480 ms	32.6 ms	91	0.03M
mm-baseline	seq32/b4096	50	1	935 ms	27.9 ms	1151	2.20M
mm-users100	seq32/b4096	100	1	5906 ms	27.2 ms	596	0.37M
mm-noprefix	prefix OFF	50	1	1440 ms	17.7 ms	1316	2.58M
mm-nochunked	chunked OFF	50	1	5520 ms	24.9 ms	745	0.27M
mm-wide-seqs	seq64/b4096	50	1	4497 ms	40.1 ms	602	0.17M
mm-wide	seq64/b8192	50	1	823 ms	24.8 ms	1159	2.84M
mm-scale2	seq32/b4096	100	2	1817 ms	18.3 ms	2298	3.46M
mm-scale4	seq32/b4096	120	4	1466 ms	23.1 ms	2854	2.52M
mm-scale4-200	seq32/b4096	200	4	4804 ms	27.1 ms	924	0.35M
mm-final	seq64/b8192	100	2	1152 ms	19.1 ms	2631	5.99M

TTFT/ITL are p95 in ms; Tput is aggregate chunks/s; score in millions. All mixed runs had 0.0% errors. The smoke run is a text-mode infrastructure check, not a track submission.

5.2 Score Ranking (mixed track)

1. mm-final 5.99M (2 GPU, seq64/b8192) - submitted 2. mm-scale2 3.46M (2 GPU) 3. mm-wide 2.84M (1 GPU, best single-GPU) 4. mm-noprefix 2.58M (1 GPU) 5. mm-scale4 2.52M (4 GPU) 6. mm-baseline 2.20M (1 GPU) 7. mm-users100 0.37M (1 GPU, saturated) 8. mm-scale4-200 0.35M (4 GPU, overloaded) 9. mm-nochunked 0.27M (1 GPU, chunked off) 10. mm-wide-seqs 0.17M (1 GPU, max-seqs starved)

6. Key Findings and Optimization Tips

6.1 Chunked Prefill Is the Most Important Mixed-Mode Knob

TIP

Keep chunked prefill ON for heterogeneous traffic. Disabling it was the worst single knob change (TTFT 935 ms -> 5.5 s at 50 users) because image/long prefills run as one step and block every queued text request. Chunking interleaves decode between prefill pieces.

6.2 max-seqs and max-batch-tokens Are Coupled

KEY FINDING

Raising max-seqs without raising max-batch-tokens starves prefill. seq64/b4096 collapsed to 0.17M (TTFT 4.5 s); seq64/b8192 was the best single-GPU run (2.84M). Scale the token budget with the concurrency, or do not raise concurrency at all.

6.3 The Mixed Workload Has a Throughput Cliff, Not a Plateau

KEY FINDING

Beyond the per-replica budget, throughput collapses rather than flattening. One GPU at 100 users dropped to 596 chunks/s; four GPUs at 200 users dropped to 924 - below the single-GPU baseline. Operate each replica near its knee (~50 users with this batch config) and add replicas for more load.

6.4 GPU Count Is a Denominator, Not a Free Win

TIP

Match load to capacity before adding GPUs. The 4-GPU run had the highest raw throughput (2854 chunks/s) but scored 2.52M - below the 2-GPU run (3.46M) - and at 200 users it collapsed to 0.35M. More GPUs help only when every replica is both saturated and fed by an adequate batch budget.

6.5 Re-measure Published Knob Guidance

TIP

The dry-run notes said seq32/b4096 beats wider settings; our data said the opposite for this workload. Vendor and reference defaults are hypotheses, not conclusions - the only authority is a measurement on your traffic and GPU.

6.6 Prefix Caching Can Cost More Than It Saves at Short Prefixes

KEY FINDING

At a ~40-token shared prompt, prefix-off slightly out-scored prefix-on (2.58M vs 2.20M) on 1 GPU because bookkeeping overhead exceeded the tiny KV reuse. We kept it on in the final (within noise at 2 replicas) but it is the top A/B candidate.

6.7 Operational Tips

TIP

Pin fastapi==0.136.0 (vLLM 0.21.0's Prometheus middleware 500s on FastAPI 0.137+). Set PYTHONUTF8=1 on Windows. Use python -m pip / python -m modal and put the user Scripts dir on PATH. Stop every app after its run (modal app stop <name> --yes) or min_containers keeps an H100 warm and billing.

7. Final Configuration and Required Answers

7.1 Final Command

```
python run_infertutor_experiment.py --label mm-final --gpu-type H100 \
  --replicas 2 --max-seqs 64 --max-batch-tokens 8192 --mode mixed \
  --users 100 --duration 90 --ramp-up 25 --max-tokens 96
```

Defaults left on: prefix caching on, chunked prefill on, eager mode, max-model-len 8192, concurrent-inputs 64, mm-max-pixels 401408. Total GPUs: 2. JSON: final_benchmark.json.

7.2 Why It Wins

- Two replicas absorb the concurrency that crushes one replica, at the most efficient point on the GPU denominator.
- The wider token budget (b8192) feeds that concurrency so prefill never starves; TTFT drops 37% versus plain 2-replica scale-out.
- Chunked prefill stays on, so image traffic does not block text decode.
- 0% errors means full goodput reaches the numerator.

7.3 Required Report Answers

- Final score: 5,985,700.
- Best TTFT p95: 823 ms (mm-wide, 1 GPU); final config 1152 ms.
- Best ITL p95: 17.7 ms (mm-noprefix, 1 GPU); final config 19.1 ms.
- Best throughput: 2854 chunks/s (mm-scale4, 4 GPU); final config 2631 chunks/s.
- Total GPU count (final): 2 x H100.
- Optimization that helped most: chunked prefill is load-bearing (its absence raised TTFT ~6x); among tuned knobs, raising max-batch-tokens (the wide batch) on 2 replicas gave the largest score gain.
- Optimization that failed/surprised: raising max-seqs alone collapsed performance; scaling to 4 replicas lowered the score and 4 GPUs at 200 users collapsed; prefix-off slightly out-scored prefix-on.
- What we would try next: Section 9.

8. vLLM Architecture Deep Dive

8.1 PagedAttention

KV cache is stored as fixed-size pages (16/32-token blocks) addressed by a block table, like OS virtual memory. This removes fragmentation, raises batch occupancy, and enables prefix sharing.

8.2 Continuous Batching

New requests are inserted into the running batch and a finished sequence's slot is reused immediately, keeping the GPU near fully utilized - the reason throughput scaled with users until the batch budget was exhausted.

8.3 Chunked Prefill

The default scheduler runs all prefills before any decode. For uniform text that is fine; for mixed traffic a 5000-token image prefill blocks everyone behind it. Chunked prefill caps each step at max-batch-tokens and interleaves decode between prefill chunks. Experiment 4.4 quantified this exactly (935 ms -> 5.5 s when disabled), and the batch-token budget (Section 4.6) is precisely the size of those chunks - which is why b8192 mattered so much.

8.4 Why Compiled Mode Is Excluded

Compiled mode pre-captures the decode loop as a CUDA graph for fixed tensor shapes - excellent for pure text. Image requests produce variable-length vision-encoder outputs that break the captured graph, so for the mixed track eager mode is the correct, stable choice.

9. What We Would Try Next

- A/B prefix-off in the final 2-replica + wide-batch config (prefix-off out-scored prefix-on on 1 GPU and was untested in combination).
- Sweep max-batch-tokens (6144, 12288) at 2 and 4 replicas - since it, not max-seqs, drove the gains, the scale-out cliff may move with a larger budget, possibly making 4 GPUs efficient at high load.
- Sweep users on 2 replicas (110, 130) to find the knee just before the cliff.
- Reduce mm-max-pixels only when image processing is the proven bottleneck (at healthy load a smaller pixel budget can worsen tensor-core efficiency).
- Map a 3-replica point to locate the exact efficiency sweet spot between 2 and 4 GPUs.

10. Conclusion

The submitted Multimodal Product configuration (mm-final, 2 x H100) scored 5,985,700 with zero errors, beating the internal 2-replica reference on ITL and throughput while using fewer GPUs than the 4-replica reference. The path mattered more than the number: we located the saturation cliff, proved chunked prefill is load-bearing, decomposed the batch knobs to show max-batch-tokens (not max-seqs, not more GPUs) was decisive, and demonstrated that adding GPUs can lower the score.

KEY FINDING

Intuition is unreliable without measurement. Prefix caching - the textbook win when requests share a prompt - slightly hurt at a 40-token prefix. Raising max-seqs - more concurrency, surely good - collapsed TTFT. Adding GPUs - the obvious latency fix - lowered the score and then collapsed at high load. Every optimization must be validated against the full scoring function, not just the metric it was meant to improve.

InferTutor Arena - Multimodal Product Track. Author: Shalini Dusharla. Inference Engineering Capstone, June 2026.